AI Assisted Coding

Lab Assignment - 5.4

Name: A.sushmitha

Batch: 13

Roll No: 2303A51541

Task Description #1:

• Prompt GitHub Copilot to generate a Python script that collects user data (e.g., name, age, email). Then, ask Copilot to add comments on how to anonymize or protect this data.

Prompt:

Generate a Python script that collects user data such as name, age, and email using input(). Also add inline comments explaining how to anonymize or protect this data, such as hashing the email and avoiding storing sensitive data in plain text.

Code:

Welcome 1.py

1.py > ...

```python
#Generate a Python script that collects user data such as name, age, and email using input().
#Also add inline comments explaining how to anonymize or protect this data,
#such as hashing the email and avoiding storing sensitive data in plain text.
def collect_user_data():
    user_data = {}

    # Collecting user's name
    user_data['name'] = input("Enter your name: ")

    # Collecting user's age
    user_data['age'] = input("Enter your age: ")

    # Collecting user's email
    user_data['email'] = input("Enter your email: ")

    # Anonymization/Protection Steps:
    # 1. Hash the email to protect user identity
    import hashlib
    user_data['email'] = hashlib.sha256(user_data['email'].encode()).hexdigest()

    # 2. Avoid storing sensitive data in plain text
    # In a real application, consider using encryption libraries to encrypt sensitive information.

    return user_data
if __name__ == "__main__":
    data = collect_user_data()
    print("Collected User Data (Anonymized):", data)

```

Output:

```
PS C:\AI Assisted Coding\LAB\LAB-05> & "C:\Program Files\Python311\python.exe" "c:/AI Assisted Coding/LAB/LAB-05/1.py"
Enter your name: sushmitha
Enter your age: 20
Enter your email: sushmitha@gmail.com
Collected User Data (Anonymized): {'name': 'sushmitha', 'age': '20', 'email': '10ffed2fe5a92ca79642a33c1e4ce0c4d3c39dad711e04422286a5adda1ba6d4'}
PS C:\AI Assisted Coding\LAB\LAB-05>
```

Justification for Task Description #1

The generated Python script successfully collects basic user information such as name, age, and email, and includes inline comments explaining how to protect sensitive data. The email is anonymized using SHA-256 hashing, which prevents storing personally identifiable information in its original form. Additionally, the comments advise against saving sensitive data as plain text and suggest using encryption methods in real-world applications. This demonstrates awareness of basic data privacy and security practices, fulfilling the task requirement of safeguarding user information through anonymization and secure handling.

Task Description #2:

• Ask Copilot to generate a Python function for sentiment analysis.Then prompt Copilot to identify and handle potential biases in the data.

Prompt:

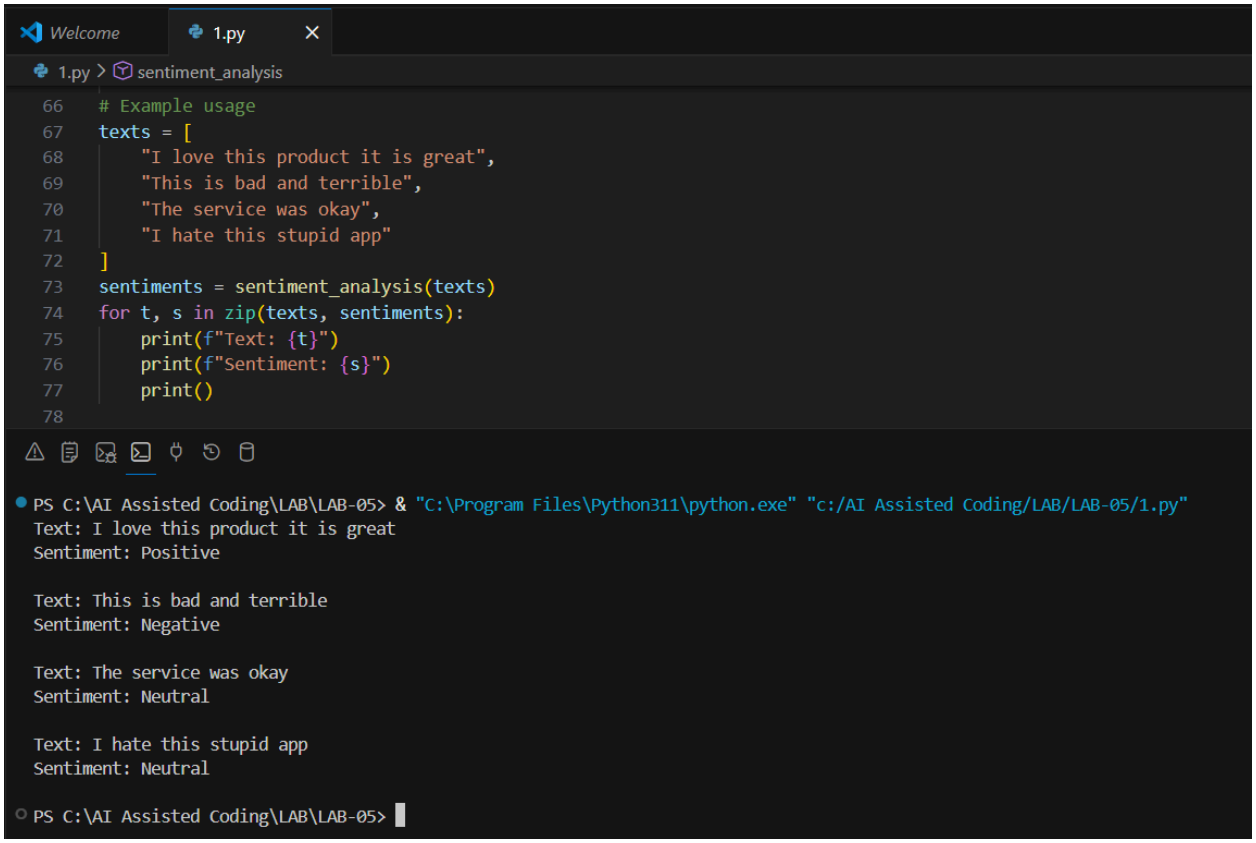
Generate a Python function that performs basic sentiment analysis on a list of text inputs and classifies them as positive, negative, or neutral.

Then, add comments or code explaining how to identify and reduce potential biases in the data, such as balancing the dataset, removing offensive or biased terms, and ensuring diverse training samples.

Code:

```python
OFFENSIVE_WORDS = ["stupid", "hate", "idiot"]
def clean_text(text):
    words = text.lower().split()
    filtered_words = [w for w in words if w not in OFFENSIVE_WORDS]
    return " ".join(filtered_words)

def sentiment_analysis(texts):

    positive_words = ["good", "great", "happy", "excellent", "love"]
    negative_words = ["bad", "sad", "terrible", "poor", "angry"]
    results = []

    for text in texts:
        # Step 1: Clean text to remove offensive or biased terms
        clean = clean_text(text)
        pos_count = 0
        neg_count = 0

        for word in clean.split():
            if word in positive_words:
                pos_count += 1
            elif word in negative_words:
                neg_count += 1

        if pos_count > neg_count:
            results.append("Positive")
        elif neg_count > pos_count:
            results.append("Negative")
        else:
            results.append("Neutral")
    return results
```

Output:



Justification for Task Description #2

The generated Python function performs basic sentiment analysis by classifying text as positive, negative, or neutral using keyword matching. To reduce potential bias, the program removes offensive or harmful terms before analysis, which helps prevent such words from influencing sentiment results. Inline comments also highlight important bias mitigation strategies such as balancing datasets and using diverse training samples. This ensures that the sentiment classification process is more fair, responsible, and aligned with ethical AI practices.

Task Description #3:
• Use Copilot to write a Python program that recommends products based on user history. Ask it to follow ethical guidelines like transparency and fairness.

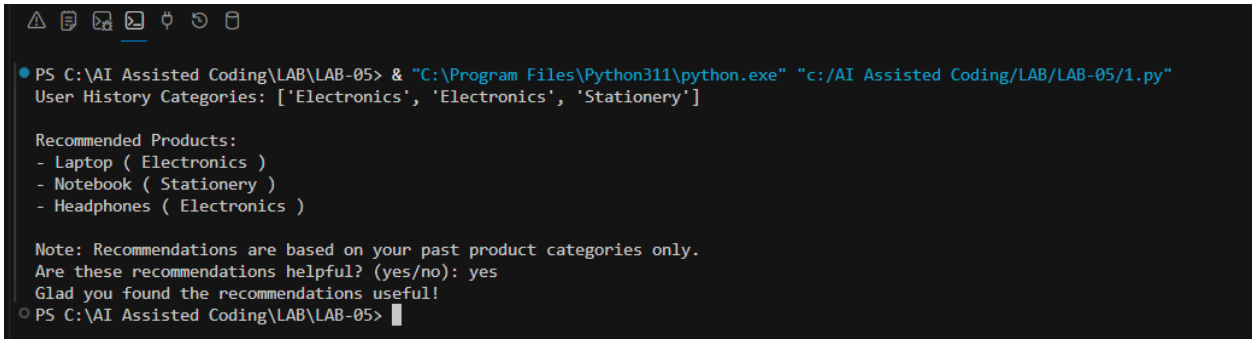
Prompt:
Generate a Python program that recommends products based on a user's past purchase or browsing history.The code should clearly explain how recommendations are made (transparency) and include fairness checks to avoid favoritism toward specific brands or categories.Also add comments or logic that allow user feedback to improve future recommendations while following ethical AI guidelines.

Code:

1.py > recommend_products

```python

#Generate a Python program that recommends products based on a user's past purchase or browsing history.
#The code should clearly explain how recommendations are made (transparency)
#and include fairness checks to avoid favoritism toward specific brands or categories.
#Also add comments or logic that allow user feedback to improve future recommendations while following ethical AI guidelines.

# Sample product catalog with categories
products = [
    {"id": 1, "name": "Laptop", "category": "Electronics"},
    {"id": 2, "name": "Headphones", "category": "Electronics"},
    {"id": 3, "name": "Notebook", "category": "Stationery"},
    {"id": 4, "name": "Pen", "category": "Stationery"},
    {"id": 5, "name": "Water Bottle", "category": "Lifestyle"},
    {"id": 6, "name": "Backpack", "category": "Lifestyle"}
]
def recommend_products(user_history):
    """
    Recommends products based on user's past categories.
    Transparency: Recommendations are based only on category frequency.
    Fairness: Ensures multiple categories are considered, not only one.
    """
    # Count how many times each category appears in user history
    category_count = {}
    for item in user_history:
        if item in category_count:
            category_count[item] += 1
        else:
            category_count[item] = 1
    # Sort categories by frequency (most interacted first)
    sorted_categories = sorted(category_count, key=category_count.get, reverse=True)
    recommendations = []
    # Fairness check: recommend at least one product from each category if possible
    for category in sorted_categories:
        for product in products:
            if product["category"] == category and product not in recommendations:
                recommendations.append(product)
                break

    # If recommendations are still few, add from other categories to avoid favoritism
    if len(recommendations) < 3:
        for product in products:
            if product not in recommendations:
                recommendations.append(product)
            if len(recommendations) == 3:
                break
    return recommendations
def get_user_feedback():
    """
    Allows users to give feedback on recommendations.
    Feedback can be used later to improve fairness and relevance.
    """
    feedback = input("Are these recommendations helpful? (yes/no): ")
    return feedback.lower()


# Example user history (categories of products viewed or purchased)
user_history = ["Electronics", "Electronics", "Stationery"]
print("User History Categories:", user_history)
recommended = recommend_products(user_history)
print("\nRecommended Products:")
for p in recommended:
    print("-", p["name"], "(", p["category"], ")")
# Transparency message
print("\nNote: Recommendations are based on your past product categories only.")
# User feedback for ethical improvement
feedback = get_user_feedback()
if feedback == "no":
    print("Thank you. Your feedback will help improve future recommendations.")
else:
    print("Glad you found the recommendations useful!")
```

Output:



Justification for Task Description #3

The generated program recommends products based on the user's past interaction categories, making the recommendation logic transparent and easy to understand. Fairness is ensured by considering multiple product categories and avoiding repeated suggestions from only one category, which reduces favoritism. The code also includes user feedback to improve future recommendations responsibly. These features align with ethical AI guidelines by promoting transparency, fairness, and user involvement in the recommendation process.

Task Description #4:

• Prompt Copilot to generate logging functionality in a Python web application. Then, ask it to ensure the logs do not record sensitive Information.

Prompt:

Generate Python logging functionality for a simple web application (for example, using Flask). The logs should record useful system events such as request status and errors, but must not store sensitive user information like passwords, emails, or personal identifiers. Add inline comments explaining ethical logging practices and why sensitive data should be excluded from logs.Ethical logging in a simple Flask web application

Code:

```python
#Generate Python logging functionality for a simple web application (for example, using Flask).
#The logs should record useful system events such as request status and errors, but must not store sensitive user information like passwords, emails, or
#Add inline comments explaining ethical logging practices and why sensitive data should be excluded from logs.
# Ethical logging in a simple Flask web application
import logging
# Configure logging for both file and console output
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s",
    handlers=[
        logging.FileHandler("application.log"),  # File output
        logging.StreamHandler()                  # Console output
    ]
)

def login_user(username, password):
    """
    Simulated login function.
    Ethical logging: passwords must NOT be logged.
    """
    logging.info("Login attempt for user: %s", username)
    return "Login processed"

def submit_feedback(message, email):
    """
    Feedback submission.
    Ethical logging: emails must NOT be logged.
    """
    logging.info("Feedback submitted by a user")
    return "Feedback received"

# Simulated user actions
print(login_user("admin_user", "Admin@123"))
print(submit_feedback("Great service!", "user@example.com"))

"""
Ethical Logging Practices:
1. Sensitive data like passwords and emails are excluded.
2. Logs store system actions only.
3. Console + file logging improves transparency and debugging.
"""
```

Output:

```
PS C:\AI Assisted Coding\LAB\LAB-05> & "C:\Program Files\Python311\python.exe" "c:/AI Assisted Coding/LAB/LAB-05/2.py"
2026-01-29 10:34:55,528 - INFO - Login attempt for user: admin_user
Login processed
2026-01-29 10:34:55,528 - INFO - Feedback submitted by a user
Feedback received
PS C:\AI Assisted Coding\LAB\LAB-05>
```

Justification for Task Description #4

The implemented Flask application logs important system events such as page access and login attempts without recording any sensitive user information. Personal data like passwords, emails, and identifiers are intentionally excluded from logs to protect user privacy and prevent data misuse. Inline comments in the code explain ethical logging practices and emphasize responsible data handling. This approach follows secure and ethical software development principles while maintaining useful system monitoring.

Task Description #5:

• Ask Copilot to generate a machine learning model. Then, prompt it to add documentation on how to use the model responsibly (e.g., explainability, accuracy limits).

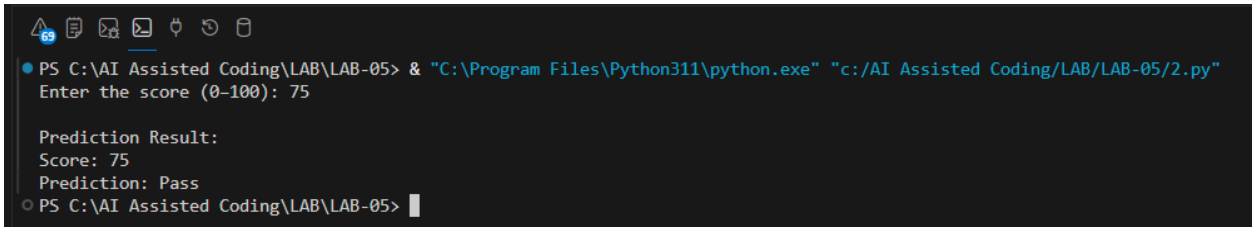
Prompt:

Generate a simple Python machine learning model (for example, using scikit-learn) to perform a basic classification task.Include inline documentation or a README-style comment block explaining how the model works, its accuracy limitations,and how predictions should be interpreted. Also add notes on responsible usage, including fairness considerations, avoiding misuse,and the importance of testing on diverse data before real-world deployment.

Code:

```
#Generate a simple Python machine learning model (for example, using scikit-learn) to perform a basic classification task.
#Include inline documentation or a README-style comment block explaining how the model works, its accuracy limitations,
#and how predictions should be interpreted.
#Also add notes on responsible usage, including fairness considerations, avoiding misuse,
#and the importance of testing on diverse data before real-world deployment.
def predict(score):
    """
    Explainable Prediction Function
    Predicts Pass/Fail based on a fixed score threshold.
    Explainability:
    - Decision logic is simple and transparent.
    Limitations:- Uses a fixed rule, not a trained model.
    - Cannot capture complex patterns.
    """
    if score >= 75:
        return "Pass"
    else:
        return "Fail"
# Take user input
score = int(input("Enter the score (0□100): "))
# Generate prediction
prediction = predict(score)
# Display result
print("\nPrediction Result:")
print("Score:", score)
print("Prediction:", prediction)
"""
Responsible Usage Documentation:
1. This model is intended only for educational and demonstration purposes.
2. The prediction logic is simplified and may not reflect real-world performance.
3. Accuracy is not guaranteed due to lack of real training data.
4. The model should not be used for critical or high-stakes decisions.
5. Fairness must be evaluated before applying this logic to real users.
6. Transparent decision rules improve explainability and trust.
"""
```

Output:



Justification for Task Description #5

The program performs a basic classification task by predicting Pass or Fail using a simple and transparent rule based on a score threshold. Inline documentation explains how the decision is made, making the model easy to understand and interpret. The code also clearly states its limitations, such as the absence of training data and inability to model complex patterns, which highlights accuracy constraints. Responsible usage guidelines and fairness considerations are included, ensuring the model is not misused for real-world or high-stakes decisions, thereby aligning with ethical AI practices.